

Nombre y apellido: LU: DNI:
E-mail: # Hojas: Firma:

Poner nombre, fecha y firma en cada hoja. Justificar todas las respuestas.

1.a	1.b	1.c	1.d	1.e	2.a	2.b	2.c	3.a	3.b	3.c	Nota

Ejercicio 1 (Programación funcional). El tipo algebraico `ABBComp` describe a los *árboles binarios de búsqueda (ABB) parcialmente comprimidos* con elementos de tipo `a` y sin repetidos. Los `ABBBComp` son árboles que pueden tener algún subárbol donde, en vez de seguir con una estructura arbórea, sus elementos se incluyen directamente en una lista ordenada (se asume que `a` pertenece a la clase `Ord`).

`data ABBComp a = Nil | Comp [a] | Nodo a (ABBBComp a) (ABBBComp a)`

Por ejemplo, las siguientes expresiones denotan árboles de tipo `ABBBComp Int` que denotan los mismos elementos:

- `Comp [0,2,14,22]`
- `Nodo 2 (Nodo 0 Nil Nil) (Comp [14,22])`
- `Nodo 14 (Nodo 2 (Nodo 0 Nil Nil) Nil) (Nodo 22 Nil Nil)`

- a) Dar el tipo y definir una función `foldABBBComp`, que abstraiga el esquema de recursión estructural sobre los *ABB parcialmente comprimidos*.
- b) Sin usar recursión explícita, dar el tipo y definir una función `mapABBBComp` que aplique una función dada a todos los valores de un árbol del tipo indicado anteriormente.
- c) Dar el tipo y definir `recABBBComp` que abstraiga el esquema de recursión primitiva sobre estos árboles.
- d) Sin usar recursión explícita, dar el tipo y definir la función `ordenado`, que permitirá verificar el invariante de la estructura (i.e., que los elementos de un `ABBBComp` estén ordenados).
- e) Dar el tipo y definir la función `iguales(a1,a2)`, que recibe dos `ABBBComp` y devuelve `True` si son iguales al recorrerlos *inorder*. Se deberá usar alguno de los esquemas de recursión anteriores y justificar por qué se lo usó.

Ejercicio 2 (Cálculo-λ). Se extiende el cálculo-λ para permitir una *maratón de términos* del mismo tipo, a partir del agregado de la extensión ganador $[M_1, \dots, M_n]$ (con $n \geq 1$). Así, el resultado de su evaluación será la del subtérmino cuya evaluación finaliza en primer lugar, yendo en rondas donde se recorren los subtérminos de izquierda a derecha (i.e., en cada ronda se evalúa un paso de cada subtérmino, si ya no es un valor, hasta recorrer todos ellos).

Por ejemplo, ganador $[if (\lambda x : bool. x) false then true else false, true, (\lambda x : bool. x) false] \rightarrow true$.

- a) Extender formalmente la gramática de los términos y el sistema de tipos.
- b) Extender la gramática de los valores, en caso de ser necesario, y las reglas de semántica operacional *small-step*. Tomar una decisión sobre lo que sucede cuando hay varios subtérminos que finalizan en la misma ronda.
De ser necesario, se pueden considerar definiciones adicionales para poder tener control de en qué paso de la evaluación se encuentra cada subtérmino. Justificar adecuadamente.
- c) Explicar detalladamente qué se modificaría en los puntos anteriores para la extensión perdedor $[M_1, \dots, M_n]$, que es igual a la anterior, pero donde el resultado de la evaluación es la del último subtérmino que finaliza su evaluación.

Ejercicio 3 (Programación Lógica). A las listas comunes de números enteros (e.g., $[8,1,1,1, 10,10,2]$) se le quiere sumar un nuevo tipo de elemento que son los comprimidos (anotados como $c(N,K)$, donde N es el número y K es la cantidad de repeticiones, con $K \geq 1$). Así, podremos generar listas *comprimidas*, donde se puedan agrupar los elementos contiguos repetidos, indicando la cantidad de valores iguales por medio un par de valores (obs.: si se comprime ciertos elementos contiguos, se deben incluir todos ellos, por lo que, por ejemplo, $[8,c(1,2), 1,10,10,2]$ no sería una compresión válida para el caso anterior). No es obligatorio comprimir toda la lista para que esta sea *comprimida*. Sólo por nombrar algunas, la lista anterior podría tener distintas versiones comprimidas: $[c(8,1), c(1,3), 10,10,2]$, $[8,c(1,3), c(10,2), 2]$, $[c(8,1), c(1,3), c(10,2), c(2,1)]$, y $[8,1,1,1,10,10,2]$.

- a) Definir un predicado `comprimido(+L1, -L2)` que dada una lista `L1`, `L2` es una versión comprimida de `L1`. No deben generarse soluciones `L2` idénticas más de una vez.
- b) ¿El predicado `comprimido` puede ser reversible en uno o ambos argumentos? Justificar detalladamente.
- c) Definir un predicado `iguales(+L1, +L2)`, que indique si dos listas son observacionalmente iguales, considerando que ambas denotan la misma lista de números enteros. Se pueden agregar predicados auxiliares.